Name : V. Dhatri Sri **Roll No.: 24501A4461**

MINI PROJECT

STUDENT PERFORMANCE PREDICTION USING MACHINE LEARNING

Abstract

Student performance is influenced by a range of academic, family, and lifestyle-related factors. This mini project applies supervised machine learning classification to predict whether a student will pass or fail based on available student information. The dataset used in the notebook is the Mathematics student performance dataset, loaded from the file named Maths.csv. The workbook contains 397 records and 33 columns before duplicate removal. Two duplicate rows were identified and removed, leaving 395 records. The final grade column, G3, was converted into a binary target: Pass for grades greater than or equal to 10 and Fail for grades below 10. G3 and the earlier grades G1 and G2 were excluded from the input features. Categorical variables were one-hot encoded and numerical variables were standardized within a scikit-learn preprocessing pipeline. Logistic Regression, Decision Tree, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM) were trained and compared. SVM obtained the highest test accuracy of 68.35%, while Decision Tree had the highest recall for the Fail class (0.50). The results show that model selection should consider the purpose of prediction, especially the need to identify students who may be at risk of failing.

1. Introduction

Educational data can be analyzed to discover patterns associated with student outcomes. Machine learning offers methods for learning relationships from historical data and applying those patterns to unseen records. In this project, student information is used to build and compare binary classification models. The purpose is educational and exploratory; predictions should not replace teacher judgement or individual support.

2. Aim

To develop and evaluate machine learning classification models that predict a student's pass/fail outcome from demographic, family, school, and lifestyle-related features in the available dataset.

3. Objectives

- Load and inspect the student mathematics performance dataset.
- Check data types, missing values, duplicate records, and summary statistics.
- Visualize the distribution of final grades and relationships between selected features and grades.
- Create a binary outcome from the final grade using a threshold of 10.
- Prepare categorical and numerical inputs for machine learning.
- Train Logistic Regression, Decision Tree, KNN, and SVM classifiers.
- Compare models using accuracy and class-specific precision, recall, and F1-score.
- Discuss limitations and possible improvements.

4. Software and Tools Required

Tool / Library	Purpose
Google Colab	Cloud-based notebook environment for running Python code.
Python	Programming language used for data analysis and modeling.
Pandas and NumPy	Data loading, cleaning, and numerical operations.

Matplotlib and Seaborn	Data visualization and model-result plots.
Scikit-learn	Preprocessing, train-test split, classifiers, and evaluation metrics.
Maths.csv	Input workbook used in the notebook; its contents were read using pandas.read_excel.

5. Theoretical Background

5.1 Machine Learning

Machine learning is a branch of artificial intelligence in which algorithms learn patterns from examples. In supervised learning, a model is trained using records that have known target labels.

5.2 Classification

Classification predicts a discrete category. Here, the two categories are Fail (0) and Pass (1). The label is derived from G3, the final grade.

5.3 Algorithms Used

Logistic Regression

A linear classification model that estimates class probabilities and is often used as an interpretable baseline.

Decision Tree

A model that splits records using feature-based rules. It can capture nonlinear relationships but may overfit.

K-Nearest Neighbors (KNN)

Classifies a record based on the labels of nearby training records; feature scaling is important.

Support Vector Machine (SVM)

Finds a decision boundary that separates classes with a large margin; scaling is important for many SVM settings.

5.4 Evaluation Metrics

Accuracy is the proportion of correct predictions. Precision measures the proportion of predicted members of a class that are correct. Recall measures the proportion of actual members of a class that are detected. The F1-score is the harmonic mean of precision and recall. For this project, Fail Recall is particularly relevant because it measures how many students who actually fail are identified by the model.

6. Methodology: CRISP-DM

The project follows the general stages of CRISP-DM (Cross-Industry Standard Process for Data Mining).

Phase	Application in this project
1. Business Understanding	Define the goal as estimating a student pass/fail outcome to support early awareness of potential academic difficulty.
2. Data Understanding	Inspect dataset shape, columns, data types, missing values, duplicates, grade distribution, and descriptive statistics.
3. Data Preparation	Remove duplicate rows, create the binary Result label, exclude G1/G2/G3 from input features, encode categorical columns, and scale numerical columns.
4. Modeling	Train Logistic Regression, Decision Tree, KNN, and SVM

	using scikit-learn pipelines.
5. Evaluation	Compare accuracy and class-specific precision, recall, F1-score, and confusion matrix.
6. Deployment	No deployment is implemented in this notebook; a future version could be exposed through a simple web interface after further validation.

7. Dataset Description

The notebook loads the file /content/Maths.csv using pandas.read_excel because the uploaded file, despite its .csv extension, was read successfully as an Excel workbook. The dataset initially contained 397 records and 33 columns. Two duplicate rows were found and removed, resulting in 395 records and 33 original columns. The dataset includes student and school characteristics, family and social attributes, study time, absences, and grade columns G1, G2, and G3.

Property	Observed value
Initial dataset shape	397 rows × 33 columns
Duplicate rows found	2
Rows after duplicate removal	395
Missing values	0 in the inspected columns
Final grade column	G3 (range 0–20)
Mean G3	10.42
Median G3	11
Standard deviation of G3	4.58
Minimum / maximum G3	0 / 20
Pass threshold	G3 ≥ 10
Pass / Fail after label creation	265 Pass (67.09%) / 130 Fail (32.91%)

Note: the source notebook identifies the file as Maths.csv and reports 395 records after duplicate removal. The report therefore describes the data actually used in the notebook rather than assuming a different record count from another version of the public dataset.

8. Procedure, Code, Outputs, and Observations

8.1 Import libraries and load the dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_excel('/content/Maths.csv')
print("Dataset loaded successfully!")
print("Number of rows and columns:", df.shape)
display(df.head())
```

Output/observation: the notebook loaded 397 rows and 33 columns. The first records contain fields such as school, sex, age, address, studytime, absences, G1, G2, and G3.

8.2 Data quality checks and duplicate removal

```
print("Missing values:", df.isnull().sum().sum())
print("Duplicates before:", df.duplicated().sum())

df.drop_duplicates(inplace=True)

print("Duplicates after:", df.duplicated().sum())
print("Updated dataset shape:", df.shape)
```

Output/observation: two duplicate rows were detected. After removal, no duplicate rows remained and the dataset shape became 395 × 33. The notebook's missing-value check returned zero missing values.

8.3 Descriptive statistics and grade distribution

```
print(df['G3'].describe())

plt.figure(figsize=(8, 5))
sns.histplot(data=df, x='G3', bins=11, kde=True)
plt.title('Distribution of Students Final Grades')
plt.xlabel('Final Grade (G3)')
plt.ylabel('Number of Students')
plt.show()
```

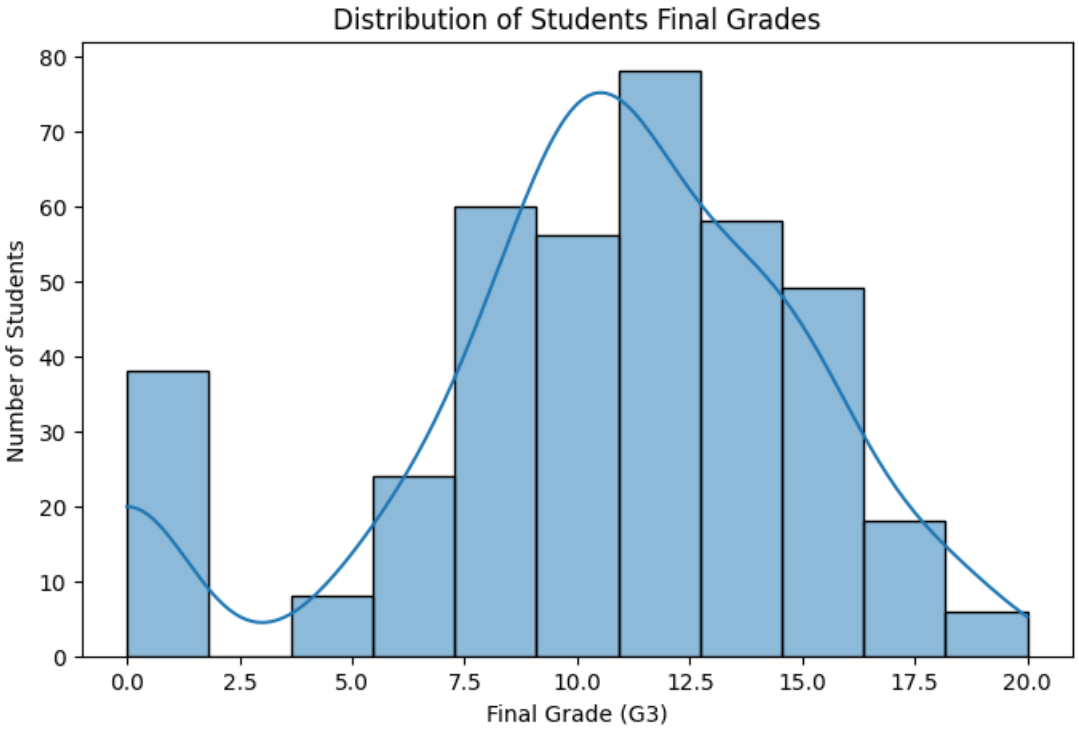


Figure 1. Distribution of final grades (G3).

Observation: G3 has a mean of approximately 10.42, a median of 11, and values between 0 and 20. The histogram shows a concentration of grades around the middle of the scale and a visible group of records with grade 0.

8.4 Explore study time and grade relationships

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='studytime', y='G3')
plt.title('Study Time vs Final Grade')
plt.xlabel('Study Time')
plt.ylabel('Final Grade')
plt.show()

plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='G1', y='G3', label='G1')
sns.scatterplot(data=df, x='G2', y='G3', label='G2')
plt.title('Previous Grades vs Final Grade')
plt.xlabel('Grade')
plt.ylabel('Final Grade')
plt.legend()
plt.show()
```

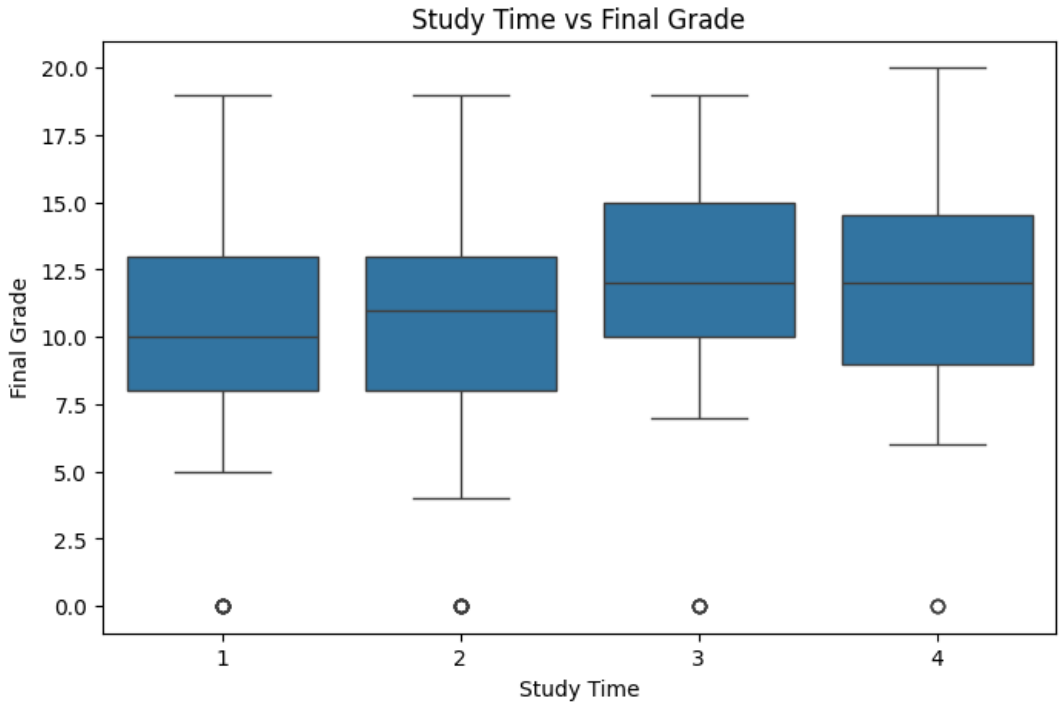


Figure 2. Study time versus final grade.

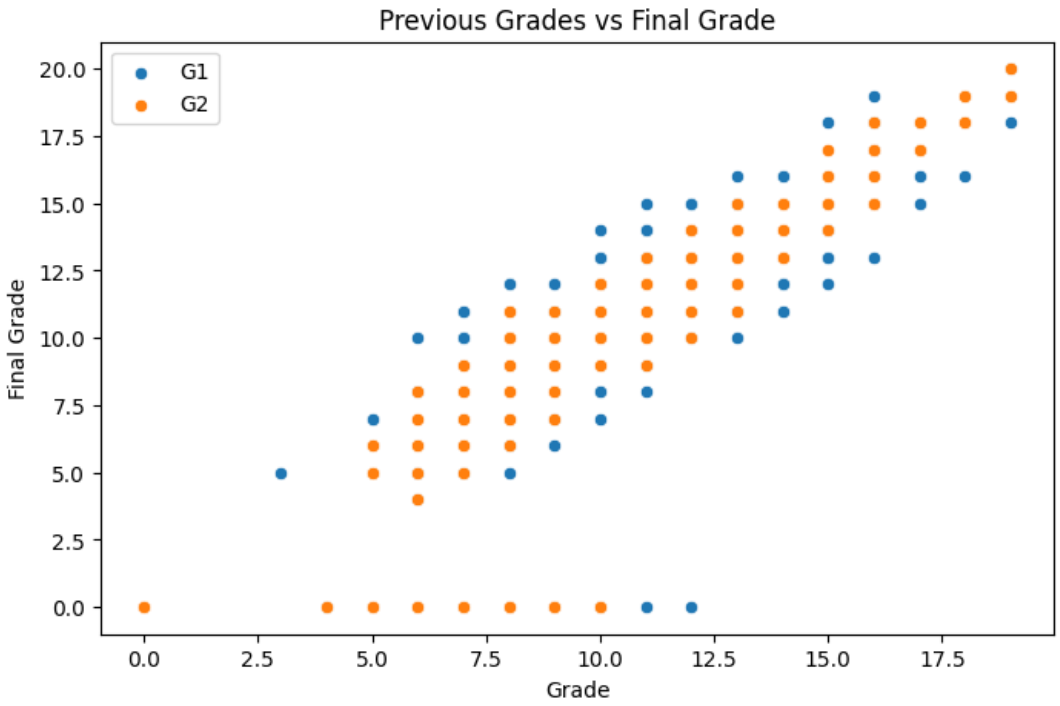


Figure 3. Previous-period grades compared with final grade.

Observation: the plots are exploratory and help inspect whether study time and previous grades are associated with final outcomes. In the predictive model, G1 and G2 are excluded to keep the task focused on predicting pass/fail from other student characteristics.

8.5 Correlation heatmap

```
numeric_df = df.select_dtypes(include='number')
sns.heatmap(numeric_df.corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()
```

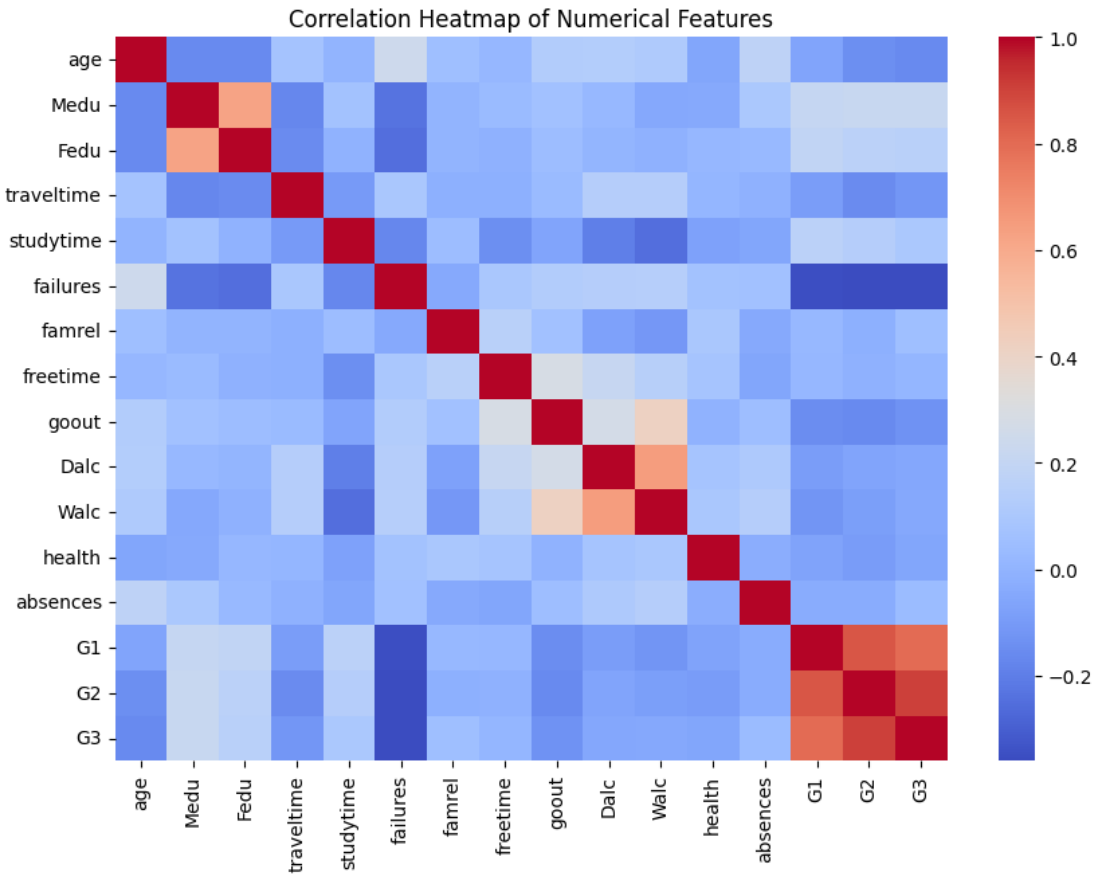


Figure 4. Correlation heatmap for numerical features.

Observation: the heatmap summarizes pairwise linear correlations among numerical columns. Correlation does not establish causation and may not capture nonlinear relationships.

8.6 Create the target variable

```
df['Result'] = (df['G3'] >= 10).astype(int)
# 1 = Pass, 0 = Fail

print(df['Result'].value_counts())
print((df['Result'].value_counts(normalize=True) * 100).round(2))
```

Output/observation: 265 records (67.09%) are labeled Pass and 130 records (32.91%) are labeled Fail. The threshold of 10 is the project's chosen rule for converting grades into two classes.

8.7 Prepare features, preprocessing, and train-test split

```
X = df.drop(columns=['G3', 'Result', 'G1', 'G2'])
y = df['Result']

categorical_cols = X.select_dtypes(
    include=['object', 'category']
).columns.tolist()
numerical_cols = X.select_dtypes(
    include=['int64', 'float64']
).columns.tolist()

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline
```

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_cols),
    ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
     categorical_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(X_train.shape, X_test.shape)
```

The final model uses all available columns except G3, Result, G1, and G2. Numerical features are standardized and categorical features are one-hot encoded. The data is split into 80% training and 20% testing data, using stratification and random_state=42 for reproducibility. The classification reports in the notebook show 79 test records.

8.8 Train and evaluate the classifiers

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=2000),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'SVM': SVC()
}

results = []
trained_models = {}

for name, model in models.items():
    pipeline = Pipeline([
        ('preprocessing', preprocessor),
        ('model', model)
    ])
    pipeline.fit(X_train, y_train)
    y_pred = pipeline.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    results.append({
        'Model': name,
        'Accuracy (%)': round(accuracy * 100, 2)
    })
    trained_models[name] = pipeline
    print(name, round(accuracy * 100, 2))
    print(classification_report(
        y_test, y_pred, labels=[0, 1],
        target_names=['Fail', 'Pass'], zero_division=0
    ))

results_df = pd.DataFrame(results).sort_values(
    by='Accuracy (%)', ascending=False
)
display(results_df)
```

9. Results and Discussion

The model comparison below uses the final comparison table shown in the submitted notebook. SVM obtained the highest accuracy, while Decision Tree had the strongest Fail Recall and Fail F1-score.

Model	Accuracy (%)	Fail Precision	Fail Recall	Fail F1-score
Decision Tree	65.82	0.48	0.50	0.49
Logistic Regression	65.82	0.47	0.35	0.40

SVM	68.35	0.55	0.23	0.32
KNN	60.76	0.31	0.15	0.21

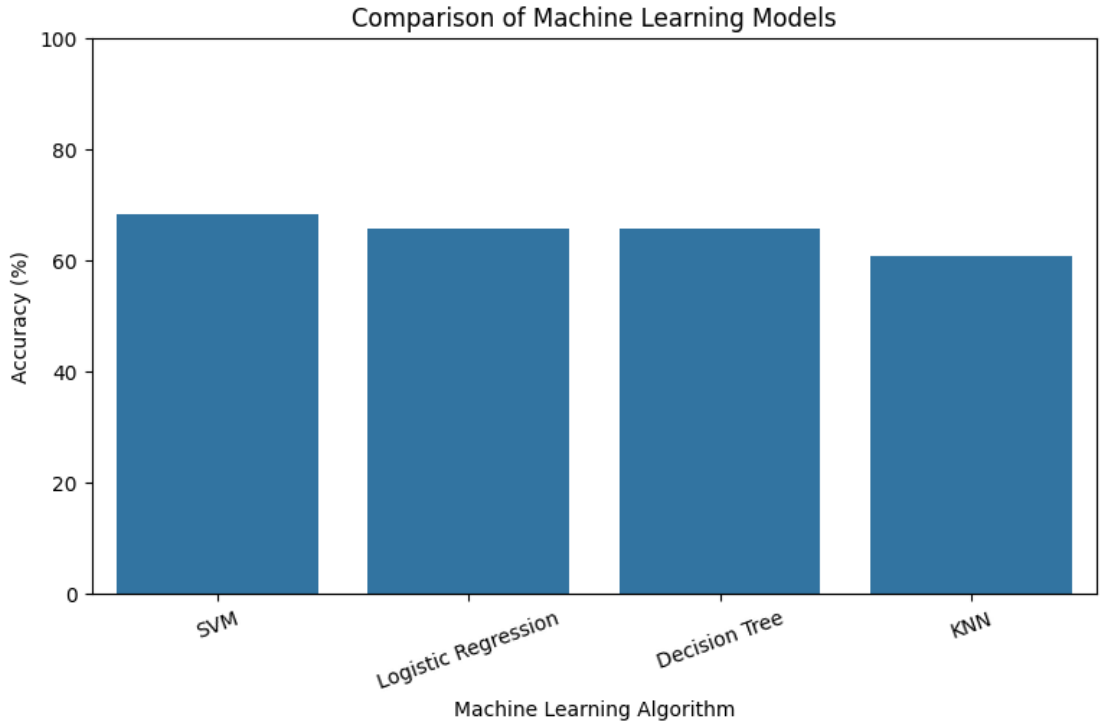


Figure 5. Accuracy comparison of the four classifiers.

SVM achieved 68.35% accuracy, the highest among the tested models. Logistic Regression and Decision Tree each achieved 65.82%, and KNN achieved 60.76%. However, Decision Tree obtained the highest Fail Recall (0.50) and Fail F1-score (0.49). Therefore, SVM is the best model by accuracy in this experiment, while Decision Tree is more promising if the main goal is to detect students who may fail.

9.1 SVM confusion matrix

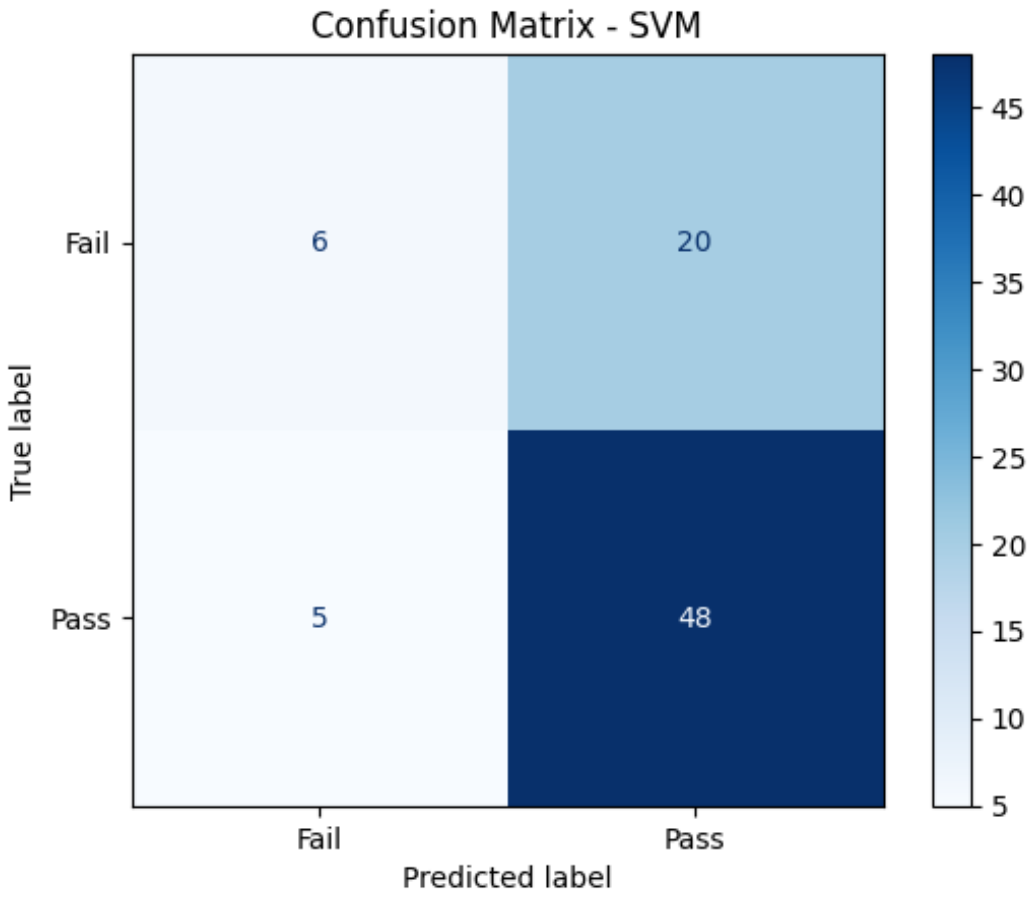


Figure 6. Confusion matrix generated in the notebook.

Actual / Predicted	Predicted Fail	Predicted Pass
Actual Fail	6	20
Actual Pass	5	48

The confusion matrix contains 79 test predictions: 6 actual failing students were correctly predicted as Fail, 20 failing students were incorrectly predicted as Pass, 5 passing students were incorrectly predicted as Fail, and 48 passing students were correctly predicted as Pass. Thus, 54 of 79 predictions were correct, giving 68.35% accuracy. The matrix agrees with the reported SVM class supports (26 Fail and 53 Pass).

9.2 SVM classification report

Class	Precision	Recall	F1-score	Support
Fail	0.55	0.23	0.32	26
Pass	0.71	0.91	0.79	53
Accuracy	—	—	0.68	79
Macro average	0.63	0.57	0.56	79
Weighted average	0.65	0.68	0.64	79

SVM recall for Pass is 0.91, meaning that it correctly identifies most passing students. Fail recall is 0.23, meaning it identifies only about 23% of students who actually fail. This is a key limitation if the model is intended to serve as an early-warning system.

10. Conclusion

This project developed a supervised machine learning workflow for predicting student pass/fail outcomes using the Mathematics student performance dataset. Data inspection and cleaning identified two duplicate records, which were removed. Exploratory analysis was performed using grade distribution, study-time plots, previous-grade plots, and a correlation heatmap. The final grade G3 was converted into a binary target using a threshold of 10. Categorical variables were one-hot encoded and numerical variables were standardized within a preprocessing pipeline. Four classifiers were trained and evaluated. SVM achieved the highest accuracy at 68.35%, whereas Decision Tree had better Fail Recall and Fail F1-score. The results demonstrate that the choice of a model should be based on the intended use case and not accuracy alone. The current performance is moderate and further improvement is needed before any real-world use.

11. Limitations

- The experiment uses a single train-test split, so results may vary with another split.
- The classification accuracy is moderate and many failing students are not detected by SVM.
- The pass/fail threshold of 10 is a project-defined assumption.
- The dataset may not include all factors that influence student performance.
- No cross-validation, hyperparameter search, or independent external validation was performed.
- The project is exploratory and does not constitute a validated student assessment system.

12. Future Scope

- Use stratified cross-validation to estimate model performance more reliably.
- Tune model parameters using GridSearchCV or RandomizedSearchCV.
- Test class-weighting or resampling methods to improve detection of the Fail class.
- Evaluate balanced accuracy, ROC-AUC, precision-recall curves, and confusion matrices.
- Compare additional models such as Random Forest and Gradient Boosting.
- Validate the workflow on another dataset before considering deployment.
- Build a simple web interface for demonstration purposes, with clear warnings about prediction uncertainty.

13. References

- Cortez, P., & Silva, A. M. G. (2008). Using data mining to predict secondary school student performance. Proceedings of the 5th Future Business Technology Conference.
- UCI Machine Learning Repository. Student Performance Data Set. https://archive.ics.uci.edu/dataset/320/student+performance
- scikit-learn documentation. https://scikit-learn.org/
- Pandas documentation. https://pandas.pydata.org/docs/
- Matplotlib documentation. https://matplotlib.org/stable/
- Seaborn documentation. https://seaborn.pydata.org/

Appendix A — Consolidated Python Code

The following consolidated code reproduces the main data preparation and model-comparison workflow from the notebook. It expects Maths.csv to be uploaded to the Colab working directory. The file is loaded with read_excel because that is the method used successfully in the supplied notebook.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    precision_score, recall_score, f1_score,
    ConfusionMatrixDisplay
)

# Load data (the supplied file has an .csv extension but is read as Excel)
df = pd.read_excel('/content/Maths.csv')
print("Initial shape:", df.shape)
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows before:", df.duplicated().sum())

# Remove duplicate records
df.drop_duplicates(inplace=True)
print("Shape after duplicate removal:", df.shape)
print("Duplicate rows after:", df.duplicated().sum())

# Explore final grades
print(df['G3'].describe())

plt.figure(figsize=(8, 5))
sns.histplot(data=df, x='G3', bins=11, kde=True)
plt.title('Distribution of Students Final Grades')
plt.xlabel('Final Grade (G3)')
plt.ylabel('Number of Students')
plt.show()

plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x='studytime', y='G3')
plt.title('Study Time vs Final Grade')
plt.xlabel('Study Time')
plt.ylabel('Final Grade')
plt.show()

plt.figure(figsize=(8, 5))
sns.scatterplot(data=df, x='G1', y='G3', label='G1')
sns.scatterplot(data=df, x='G2', y='G3', label='G2')
plt.title('Previous Grades vs Final Grade')
plt.xlabel('Grade')
plt.ylabel('Final Grade')
plt.legend()
plt.show()

numeric_df = df.select_dtypes(include='number')
plt.figure(figsize=(10, 7))
sns.heatmap(numeric_df.corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap of Numerical Features')
plt.show()

# Target: 1 = Pass, 0 = Fail
df['Result'] = (df['G3'] >= 10).astype(int)
print(df['Result'].value_counts())

# Exclude final and earlier grades from predictors
X = df.drop(columns=['G3', 'Result', 'G1', 'G2'])
y = df['Result']
```

```
categorical_cols = X.select_dtypes(
    include=['object', 'category']
).columns.tolist()
numerical_cols = X.select_dtypes(
    include=['int64', 'float64']
).columns.tolist()

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numerical_cols),
    ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
     categorical_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

models = {
    'Logistic Regression': LogisticRegression(max_iter=2000),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'SVM': SVC()
}

results = []
trained_models = {}

for name, model in models.items():
    pipeline = Pipeline([
        ('preprocessing', preprocessor),
        ('model', model)
    ])
    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_test)
    trained_models[name] = pipeline

    results.append({
        'Model': name,
        'Accuracy (%)': round(accuracy_score(y_test, predictions) * 100, 2),
        'Fail Precision': round(precision_score(
            y_test, predictions, pos_label=0, zero_division=0), 2),
        'Fail Recall': round(recall_score(
            y_test, predictions, pos_label=0, zero_division=0), 2),
        'Fail F1-score': round(f1_score(
            y_test, predictions, pos_label=0, zero_division=0), 2)
    })

    print('\n', name)
    print(classification_report(
        y_test, predictions, labels=[0, 1],
        target_names=['Fail', 'Pass'], zero_division=0
    ))

results_df = pd.DataFrame(results).sort_values(
    by='Accuracy (%)', ascending=False
)
print(results_df)

# Accuracy comparison plot
plt.figure(figsize=(9, 5))
sns.barplot(data=results_df, x='Model', y='Accuracy (%)')
plt.title('Comparison of Machine Learning Models')
plt.xlabel('Machine Learning Algorithm')
plt.ylabel('Accuracy (%)')
plt.xticks(rotation=20)
plt.ylim(0, 100)
plt.tight_layout()
plt.show()

# Best model by accuracy
best_model_name = results_df.iloc[0]['Model']
best_model = trained_models[best_model_name]
y_pred_best = best_model.predict(X_test)

cm = confusion_matrix(y_test, y_pred_best, labels=[0, 1])
ConfusionMatrixDisplay(
```

```
    confusion_matrix=cm, display_labels=['Fail', 'Pass']
).plot(cmap='Blues')
plt.title('Confusion Matrix - ' + best_model_name)
plt.show()


print("Best model:", best_model_name)
print("Confusion matrix:\n", cm)
print(classification_report(
    y_test, y_pred_best, labels=[0, 1],
    target_names=['Fail', 'Pass'], zero_division=0
))
```